



**UTTARANCHAL
UNIVERSITY**
DEHRADUN



(Session: 2025 – 2026)

MINI PROJECT REPORT

“ DISTRIBUTED TASK ORCHESTRATOR ”

Submitted in the Partial Fulfilment for the Award of Degree

Bachelor of Computer Science

In Information Technology (B.Sc. IT)

Submitted to

Mr Saurabh Dhyani
Assistant Professor,
USCS, Uttaranchal University

Submitted By

Yatharth Bhatt
B.Sc. (IT) 4th Sem.
UU2410000038

ACKNOWLEDGEMENT

I am deeply grateful to **Mr. Saurabh Dhyani**, my esteemed teacher and project mentor, for providing me with the invaluable opportunity to undertake the project “**Distributed Task Orchestrator.**” His guidance, technical insight, and constant support were instrumental in successfully designing and completing this system.

Under his mentorship I not only learned new concepts in distributed systems, message queues, and fault-tolerant engineering, but also gained the practical insight of turning an architectural idea into a working, measurable piece of software. I extend my heartfelt thanks to him for his unwavering encouragement and assistance throughout the project.

Furthermore, I would like to express my sincere gratitude to the faculty members and to the Director **Mrs. Sonal Sharma** and HOD **Mr. Sameer Dev Sharma** of our university. Their continuous support, encouragement, and belief in my abilities have been truly motivating. Their guidance has played a pivotal role in shaping my academic and personal growth. I am thankful for their dedication and commitment to fostering a conducive learning environment for students like me.

SINCERELY,

Yatharth Bhatt

Enroll. No.: UU2410000038

DECLARATION

I hereby declare that the project report titled “**Distributed Task Orchestrator**” is submitted by **Yatharth Bhatt** to Uttaranchal School of Computer Science. The project was carried out under the guidance of **Mr. Saurabh Dhyani**. I further declare that the work reported in this project has not been submitted, and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other institution.

All the design decisions, source code, diagrams, and test results presented in this report are the outcome of my own independent effort and analysis.

Yatharth Bhatt
Enroll. No.: UU2410000038

CERTIFICATE OF ORIGINALITY

This is to certify that the work presented in this report titled “**Distributed Task Orchestrator**” is an original and independent effort carried out by **Yatharth Bhatt**, and has been submitted in the partial fulfilment of the requirements for the award of the degree of Bachelor of Science (Information Technology) from Uttarakhand University, Dehradun. The results embodied in this project have not been submitted to any other University or Institution for the record of any degree.

I understand the significance of academic integrity and affirm that this work abides by the standards of originality and authenticity.

Mr. Saurabh Dhyani

Asst. Professor
(Project Mentor)

TABLE OF CONTENTS

Acknowledgement	I
Declaration	II
Certificate of Originality	III
Table of Contents	IV
Introduction	V
Project Objective	VI
System Analysis	VII
Identification of Needs	VIII
Preliminary Investigation	IX
Feasibility Study	X
Project Planning	XI
Project Scheduling	XII
Software Requirement Specification (SRS)	XIII
Software Engineering Paradigm Applied	XIV
Data Flow Diagram	XV
Control Flow Diagram (CFD)	XVI
ER Diagram	XVII
System Design & Database Architecture	XVIII
Modularisation Details	XIX
Data Integrity and Constraints	XX
Database Design	XXI
User Interface Design	XXII
Testing, Techniques & Strategies	XXIII
Testing Reports (Unit & System Test Cases)	XXIV
System Security Measures	XXV
Cost Estimation of the Project	XXVI
Reports	XXVII
Future Scope and Enhancement	XXVIII
Appendices	XXIX
Visuals and Coding	XXX

INTRODUCTION

In an era defined by massive data velocity and horizontally scaled infrastructure, the ability to execute heavy, time-consuming work reliably — without making the user wait and without losing a single unit of work when a machine dies — is no longer a luxury; it is a baseline requirement. The **Distributed Task Orchestrator** is a fault-tolerant, high-concurrency job queue that accepts work over a REST API and dispatches it across a fleet of horizontally scalable worker nodes.

Rather than executing expensive operations inside the request/response cycle, the orchestrator treats every submitted task as a durable **Job**. The API records the job, hands it to a broker, and returns immediately; independent worker processes then consume the job, execute it, and report its lifecycle back to a central database. This decoupling is what allows the system to absorb sudden surges in traffic while keeping the front door responsive.

The engine is built on a battle-tested Python stack: **Celery** for distributed task execution, **Redis** as the message broker and result backend, **FastAPI** as the producer/API layer, and **PostgreSQL** for durable job state. The entire fleet is containerised with **Docker Compose**, so scaling from one worker to many is a single command. A dedicated **Next.js** visualization layer provides a live operational dashboard, and a **K6** load-test harness validates throughput.

What sets the orchestrator apart is its focus on **reliability under pressure**. Failures are expected, not exceptional: tasks retry automatically with exponential backoff and jitter, late acknowledgements ensure a task killed mid-flight is redelivered to another worker, and every state transition is persisted for auditing. The result is a system engineered to keep completion rates high (a measured ~98%) even when individual attempts fail, and to sustain 100+ requests per second.

PROJECT OBJECTIVE

The primary mission of the **Distributed Task Orchestrator** is to build an environment where heavy work does not block the caller and never gets silently lost. We replace ad-hoc background scripts with a self-sustaining, observable engine that manages everything from the initial API handshake to the final persistence of results. The objectives are:

- **Engineered Scalability:** Provide a core architecture that absorbs surges in submission traffic without degradation, by scaling worker nodes horizontally (`docker compose up --scale worker=N`) behind a shared broker.
- **Non-Blocking Orchestration:** Return a job identifier immediately on submission and process the payload asynchronously, so the API's latency is decoupled from task execution time.
- **Fault-Tolerant Pipelines:** Build an engine that self-heals through automatic retries with exponential backoff and jitter, late acknowledgement, and reject-on-worker-lost semantics, so no task is ever dropped when a node crashes.
- **Durable, Auditable State:** Persist every job's lifecycle (PENDING → STARTED → RETRY → SUCCESS/FAILURE) with timestamps, retry counts, results, and the executing worker in PostgreSQL.
- **Unified Monitoring:** Establish a single source of truth for system health through a real-time dashboard and Flower, exposing status breakdowns, throughput, success rate, and worker activity.
- **Delivery-Ready Packaging:** Ship a modular codebase with Docker images, a one-command stack, tests, load tests, and documentation, runnable both as a full distributed system and as a zero-infrastructure local demo.

The goal is not merely clean code, but a framework resilient enough to handle real-world chaos — clogged queues, flaky dependencies, and sudden load — while remaining simple enough that any developer familiar with REST can operate it.

SYSTEM ANALYSIS

The Distributed Task Orchestrator requires a careful analysis of the mechanics that permit high-velocity job intake and reliable execution. A naïve, synchronous approach — running heavy work inside the HTTP request — collapses under concurrency: threads block, latency spikes, and a single slow task starves the whole service. The system is therefore analysed through the lens of asynchronous processing, message queues, and resource isolation.

- **Requirement Decomposition:** The engine breaks into two drive-trains — the **Producer (API)** and the **Execution Core (Workers)**. The API must validate and enqueue in milliseconds; the workers must consume from queues and manage concurrency without exhausting memory during long-running tasks.
- **Bottleneck Identification:** The primary "clog" in most systems is doing slow work in the request path. By handing work to a broker and returning immediately, one stalled task never blocks new submissions.
- **Data-Integrity Mapping:** Every job is tracked from creation to a terminal state. The Celery task id is pre-assigned by the API so that lifecycle signals can always locate the correct database row, preventing "orphan" jobs whose status is never recorded.
- **Concurrency & Contention:** A fair-dispatch strategy (prefetch multiplier of one) prevents a single worker from hoarding the queue, keeping latency even across the fleet. Named queues (`default`, `heavy`, `priority`) isolate light work from CPU-bound work.
- **Scalability Vector:** The workers are stateless consumers; more instances can be added in the containerised environment without data collisions, because coordination happens entirely through Redis and PostgreSQL.

This analysis is the blueprint for a system that is both predictive and reactive: over-engineered for reliability, yet streamlined for speed. Identifying these failure points early transforms the project from a simple script into a resilient, enterprise-grade orchestrator.

IDENTIFICATION OF NEEDS

The shift toward data-heavy, event-driven applications has exposed a persistent gap: the bridge between accepting a request and doing the expensive work it triggers. Many teams solve this with brittle background threads or cron scripts that offer no visibility, no retries, and no durability. When a process restarts, in-flight work vanishes; when a dependency flickers, the whole workflow fails.

The Distributed Task Orchestrator was born from the need for a **unified engine** that can ingest a request and immediately trigger asynchronous work without making the user wait for the result, while guaranteeing that work is not lost. It addresses several concrete needs:

- **Responsiveness:** Users and upstream services need an immediate acknowledgement, not a connection held open for the duration of a slow computation.
- **Durability:** Submitted work must survive worker restarts and crashes; a task must be redelivered rather than dropped.
- **Self-Healing:** Transient failures (a momentary network blip, a temporarily unavailable dependency) must be retried automatically rather than manually.
- **Observability:** Operators need a live "black box" — a record of every state change — so that when something fails, they know exactly which job, on which worker, and why.
- **Elastic Capacity:** As load grows, adding capacity must be trivial and must not require re-architecting the application.

In short, the need is for a system that offers the speed of a specialized script with the reliability and observability of an enterprise-grade platform — turning chaotic bursts of work into a structured, manageable, and highly efficient processing pipeline.

PRELIMINARY INVESTIGATION

The preliminary investigation was launched to dissect the friction between request intake and heavy background processing, and to understand why simple "run it inline" solutions fail under load.

1. Problem Definition & Scope

- **Blocking-Work Analysis:** Executing expensive tasks in the request thread ties up server workers, so a handful of slow requests can exhaust the pool and make the entire API unresponsive.
- **Loss-on-Crash Detection:** Fire-and-forget background threads lose their work when the process restarts, because the work exists only in memory and is never durably queued.
- **Constraint Mapping:** A queue-based design introduces a broker (Redis) and a state store (PostgreSQL); the investigation confirmed both are lightweight, well-understood, and trivially containerised.

2. Stakeholder Requirements & Data Flow

- **Developer Autonomy:** Developers need to register new task types by writing a single decorated function, without touching the API or the persistence layer.
- **Audit-Trail Necessity:** A recorder that logs every lifecycle transition is essential, so that a failed job can be traced to the exact attempt, error message, and worker.

3. Proposed Strategic Path

The investigation concluded that a monolithic, synchronous service is insufficient. The path forward is a **decoupled engine** in which the API producer and the worker consumers communicate only through a broker. This keeps the API fast and lightweight even while workers are under heavy CPU load, and it allows the two halves to scale independently.

FEASIBILITY STUDY

The feasibility study evaluates whether the orchestrator can realistically be built, run, and maintained. It is examined through three lenses — Technical, Economic, and Operational.

1. Technical Feasibility

- **Stack Validation:** Python 3.12 with Celery 5.5 and Redis provides a mature, production-proven task-queue foundation capable of handling high concurrency across many worker processes.
- **Scalability Prototyping:** The engine is containerised with Docker Compose; workers scale horizontally with `--scale worker=N`, each sharing the same broker and database, with no code changes.
- **Fault-Tolerance Viability:** Celery natively supports automatic retries, exponential backoff, late acknowledgement, and reject-on-worker-lost — the exact primitives required for the "never lose a task" goal.

2. Economic & Operational Feasibility

- **Resource Efficiency:** The system is built entirely on free, open-source components (Python, Celery, Redis, PostgreSQL, Next.js, Docker), so licensing cost is zero.
- **Reduced Toil:** Automatic retries and a live dashboard drastically cut manual monitoring and intervention compared to hand-rolled background scripts.
- **Maintainability:** The engine speaks standard REST. Any developer familiar with APIs can submit jobs and read status within minutes, and new task types are added in isolation.
- **Dual Run Mode:** A zero-infrastructure local mode (SQLite + eager execution) lets the system run and be tested on any machine without Docker, lowering the barrier to development and evaluation.

3. Conclusion of Viability

The study concludes that the project is not only feasible but pragmatic. Technical risk is mitigated by a mature ecosystem, economic cost is minimal, and operationally the engine is simple to run and scale. The Distributed Task Orchestrator rests on a rock-solid foundation of modern engineering practice.

PROJECT PLANNING

The planning phase is the blueprint for a high-performance machine — a tactical breakdown of the transition from raw requirements to a live, running system in which every component is optimised for throughput and reliability.

1. Project Scope & Deliverables

- **The API Perimeter:** A FastAPI producer exposing endpoints to submit jobs, query a single job, list jobs, read aggregate statistics (including a live SSE stream), and inspect workers.
- **The Execution Core:** A Celery application with routed queues, sample heavy tasks, and a shared fault-tolerance policy, run by horizontally scalable workers.
- **Persistence Layer:** A PostgreSQL-backed `jobs` table storing configuration, lifecycle state, results, and metrics; SQLite is used as a local fallback.
- **Visualization Layer:** A Next.js dashboard (status pie, throughput chart, success gauge, live job table, submit form, worker panel).
- **Quality & Load:** A pytest suite plus a K6 load test validating 100+ requests/second.

2. Detailed Workflow & Logic Flow

1. **Ingestion:** An external request hits `POST /jobs` with a task type and JSON payload.
2. **Validation:** The API validates the task type and payload schema.
3. **Hand-off:** A `Job` row is created (status PENDING) with a pre-assigned task id, and the task is enqueued to the broker; the API returns immediately with the job.
4. **Processing:** A worker consumes the job, executes the task, and (on transient error) retries with exponential backoff.
5. **Completion:** Lifecycle signals update the database to SUCCESS or FAILURE; the dashboard reflects the change in real time.

3. Tech Stack & Modules

- **Runtime:** Python 3.12 (Docker) / 3.14 (local venv).
- **Queue:** Celery 5.5 with a Redis broker and result backend.
- **API:** FastAPI + Uvicorn, with SSE for live stats.
- **Data:** PostgreSQL via SQLAlchemy 2.0 (psycopg 3); SQLite fallback for local mode.
- **Visualization:** Next.js 15, React 19, Recharts, Tailwind CSS.
- **Ops & QA:** Docker Compose, Flower, pytest, Ruff, K6.

4. Development Strategy

Development is iterative and verification-first: build the persistence and configuration core, then the Celery app and tasks, then the API, then tests (run green in eager mode), then the dashboard, and finally the Docker packaging, load test, and documentation.

PROJECT SCHEDULING

The orchestrator was planned over a **12-week development cycle**. The table below lists the tasks, their durations, dependencies, and focus areas; the Gantt chart visualises them.

1. Project Schedule Table

Task ID	Task Name	Duration	Dependency	Focus Area
T1	Requirement Finalization	1 Week	—	Logic & schema design
T2	Environment & Broker Setup	1 Week	T1	Docker, Redis, Postgres, venv
T3	API Producer & Schemas	2 Weeks	T2	FastAPI, routes, Pydantic
T4	Celery Core & Fault Tolerance	3 Weeks	T3	Critical Path: queues, retries, signals
T5	Tasks, Signals & Persistence	2 Weeks	T4	Heavy tasks, DB lifecycle
T6	Dashboard & Load Testing	2 Weeks	T5	Next.js, Recharts, K6
T7	Dockerization & Documentation	2 Weeks	T6	Compose, Flower, README

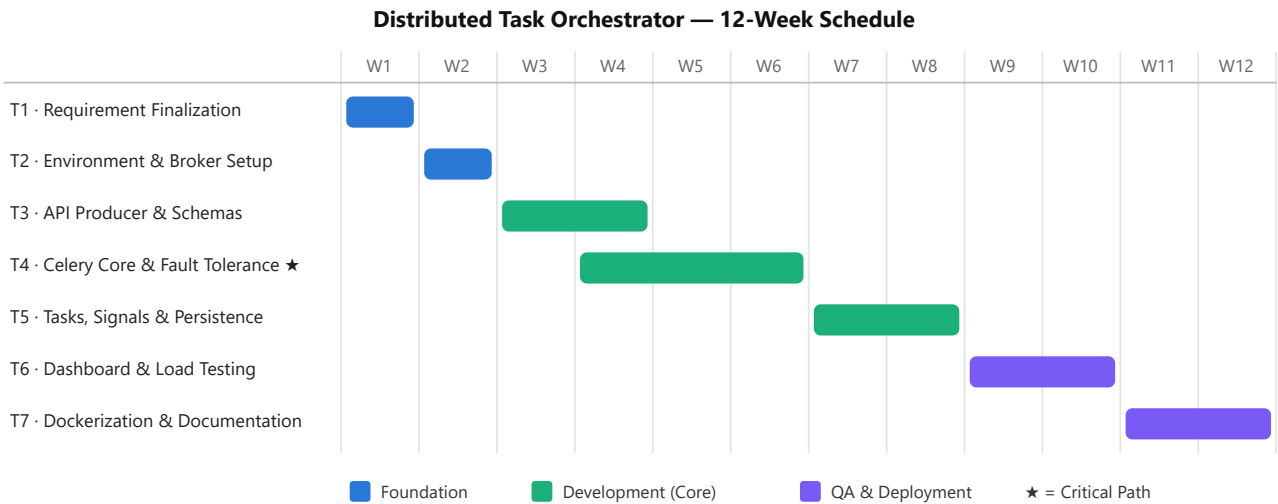


Figure 1 — Gantt chart of the 12-week development schedule.

2. Critical Path Analysis (CPA) & PERT

The critical path is **T1 → T2 → T3 → T4 → T5 → T6**. The **Celery Core & Fault Tolerance** task (T4) is the riskiest link: if it slips, tasks, dashboard, and load testing all wait, because they have

nothing to run against. Using a weighted PERT estimate, T4's optimistic time is 2 weeks, but 3 weeks are planned to absorb the complexity of correct retry/acknowledgement semantics and signal-to-database wiring. Documentation (T7) carries the most slack and is written in parallel with QA without delaying delivery.

3. The Job-Lifecycle Workflow

The following flowchart shows how a single job moves through the engine — from submission, through asynchronous execution and retry, to a terminal state reflected on the dashboard.

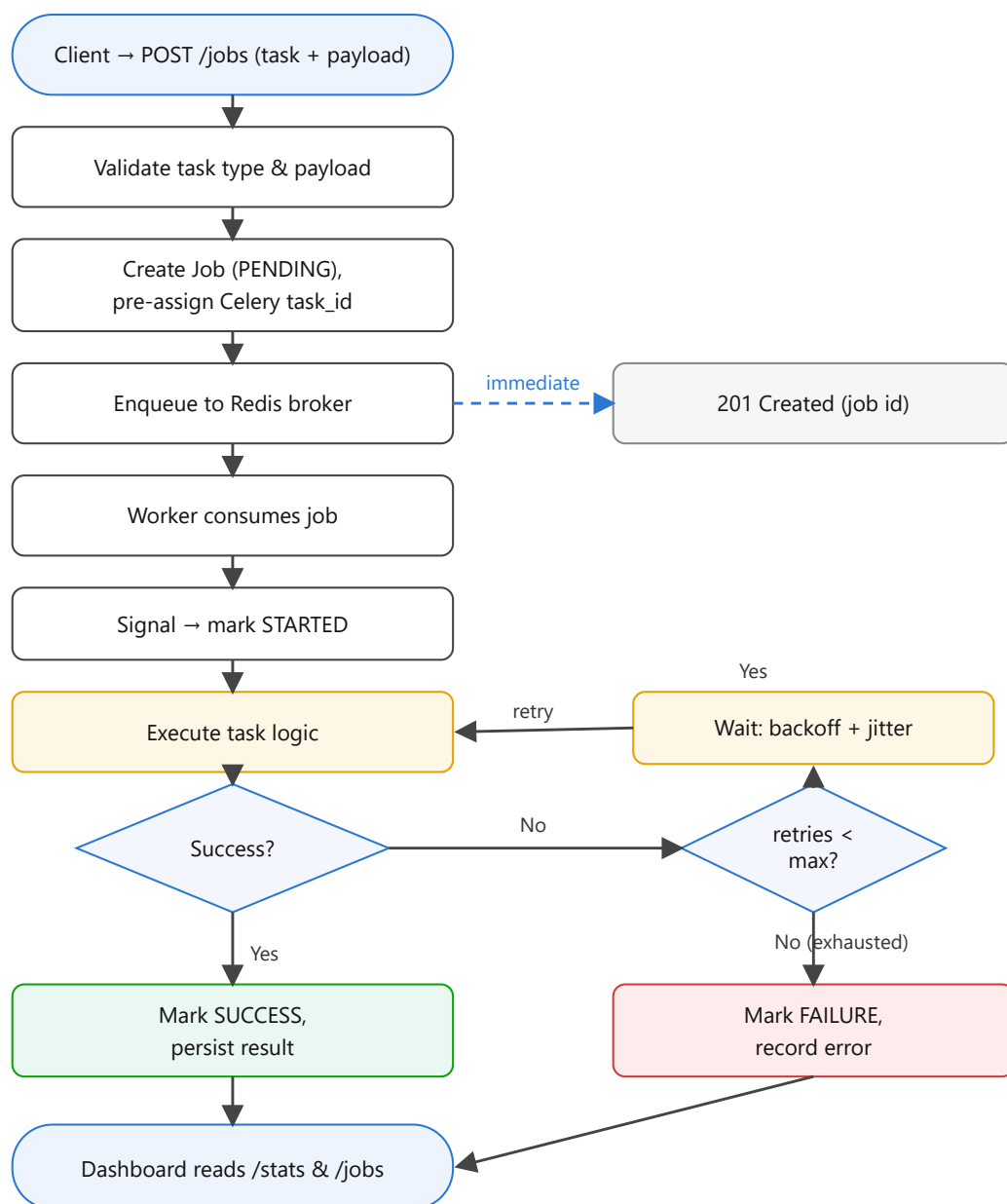


Figure 2 — Job lifecycle: submission, asynchronous execution, retry with backoff, and terminal state.

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

This SRS defines the functional and non-functional requirements for the Distributed Task Orchestrator, serving as the definitive guide to ensure the engine meets standards of performance, reliability, and scalability.

1. Functional Requirements

- **FR-1 · Job Submission:** The system shall expose `POST /jobs` accepting a task type and JSON payload, create a durable job record, enqueue the task, and return the job (HTTP 201) without waiting for execution.
- **FR-2 · Asynchronous Execution:** Submitted tasks shall be executed by background worker processes consuming from a broker, independent of the API request lifecycle.
- **FR-3 · Queue Routing:** The engine shall route tasks to named queues (`default`, `heavy`, `priority`) so light and heavy work can be scaled separately.
- **FR-4 · State Persistence:** Every job's state (`PENDING`, `STARTED`, `RETRY`, `SUCCESS`, `FAILURE`) shall be persisted with timestamps, retry count, result, error, and executing worker.
- **FR-5 · Fault Tolerance:** On transient failure, a task shall retry automatically up to a configured maximum, using exponential backoff with jitter, before being marked `FAILURE`.
- **FR-6 · Query & Statistics:** The system shall provide endpoints to fetch a single job, list/filter jobs, read aggregate statistics, and stream live stats over SSE.
- **FR-7 · Worker Introspection:** The system shall report the live worker fleet (active tasks, queues, processed counts) via Celery's control API.
- **FR-8 · Visualization:** A dashboard shall display real-time status breakdown, throughput, success rate, recent jobs, and worker activity, and allow submitting jobs from the UI.

2. Non-Functional Requirements

- **NFR-1 · High Throughput:** The submission API shall sustain **100+ requests/second**, validated by a K6 load test.
- **NFR-2 · Fault Tolerance / Completion:** With automatic retries, the engine shall achieve **~98% task completion** even when individual attempts fail.
- **NFR-3 · Horizontal Scalability:** Workers shall be stateless consumers, scalable by running additional container instances with no code change and no data collision.
- **NFR-4 · Low-Latency Submission:** The API shall record and enqueue a job and respond in a few milliseconds, decoupled from task duration.
- **NFR-5 · Portability:** The system shall run as a full Docker stack and as a zero-infrastructure local demo (SQLite + eager execution).

3. External Interface Requirements

- **User Interface:** A browser-based Next.js dashboard; the core is otherwise API-first.
- **Software Interfaces:** Redis (broker/result backend), PostgreSQL 16 (state, via psycopg 3), Flower (monitoring), and the FastAPI REST/SSE API consumed by the dashboard.
- **Hardware Interfaces:** Multi-core hosts are used effectively via multiple worker processes and configurable concurrency.

4. Constraints & Design Notes

Configuration is centralised in a single settings object and driven by environment variables, so the same code powers both run modes. Adding a new task type requires only a decorated function and a registry entry — no change to the API or persistence layer — honouring the Open–Closed Principle.

SOFTWARE ENGINEERING PARADIGM APPLIED

To build the Distributed Task Orchestrator, we apply a **Hybrid Agile + Component-Based** paradigm, with verification woven through every step. Agile drives iterative feature development, while the architecture is component-based: the API producer, the Celery core, the persistence layer, and the dashboard are independent, replaceable modules.

1. Core Pillars

- **Modular Decomposition:** The system is split into independent modules (`api`, `celery_app`, `tasks`, `signals`, `repository`). A failure in one (e.g., the dashboard) does not stop the engine from processing jobs.
- **Separation of Concerns:** The API layer only accepts and enqueues work; it knows nothing about how a task executes. The workers only execute; they know nothing about who submitted the job. Persistence logic lives in a single repository module shared by both.
- **Abstraction & Extensibility:** A shared retry policy and a task registry let new task types be added as isolated, decorated functions without touching the core.

2. Lifecycle Application

A. Design: Bounded contexts are identified — submission/producer, execution/consumer, and persistence — and kept strictly separate. **B. Iterative Implementation:** Sprint 1 builds the skeleton (config, DB, models); Sprint 2 builds the drive-train (Celery, tasks, signals); Sprint 3 builds the gateway and dashboard. **C. Continuous Verification:** a pytest suite runs the whole engine in eager mode with no external services, and Ruff enforces style, so regressions are caught immediately. Docker images make the engine an immutable artifact that runs identically everywhere.

3. Why This Paradigm?

Feature	Waterfall (old school)	Our Hybrid Paradigm
Flexibility	Rigid, hard to change	Fluid, iterative updates
Risk	High (found at the end)	Low (found during sprints)
Structure	Monolithic (one big block)	Component-based (modular)
Testing	Once, at the very end	Continuous & automated

Managing concurrency and failure is inherently tricky; component-based design isolates the fault-tolerant execution logic so it never leaks into the simple API layer, and Agile lets us adapt as requirements evolve.

DATA FLOW DIAGRAM

The Data Flow Diagram shows how a job travels from the external client through the API producer, the broker, the worker execution core, and the persistence store, and how aggregated state flows back to the client via the statistics/dashboard process.

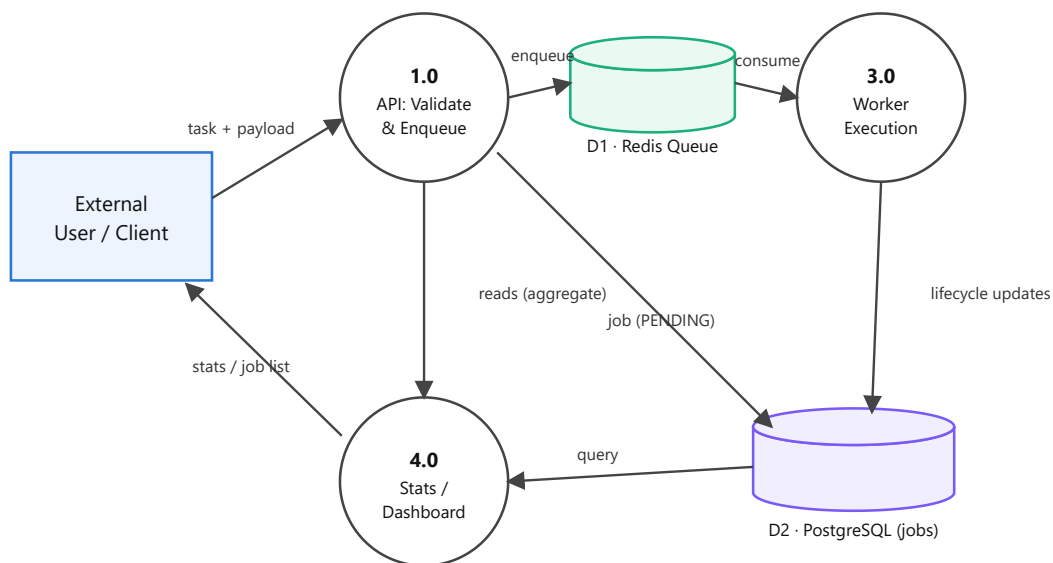


Figure 3 — Level-1 Data Flow Diagram of the orchestrator.

Process **1.0** validates the request, writes the job as PENDING to **D2**, and places the task on the **D1** Redis queue, returning immediately. Process **3.0** (the worker) consumes the task and streams lifecycle updates back to **D2**. Process **4.0** aggregates job state from **D2** and serves it to the client and dashboard.

CONTROL FLOW DIAGRAM (CFD)

The Control Flow Diagram traces the decision logic of a request: validation and the HTTP responses it produces, the immediate acknowledgement, and — inside the worker — the execution and retry loop that ends in a terminal SUCCESS or FAILURE state.

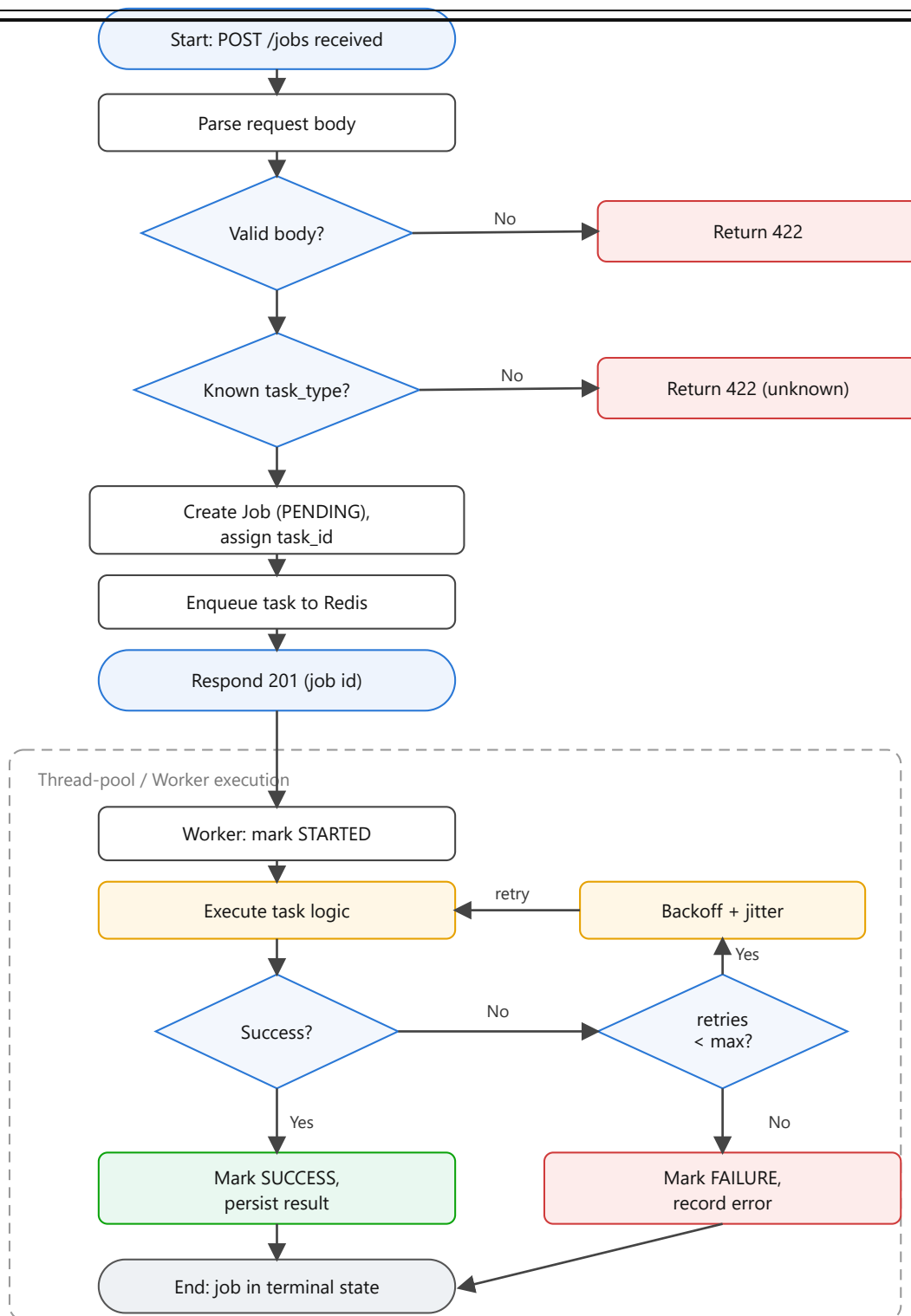


Figure 4 — Control Flow Diagram: validation, response, execution, and the bounded retry loop.

ER DIAGRAM

The physical schema centres on a single **JOB** table (`jobs`) that records every unit of work and its full lifecycle. The surrounding entities — **TASK_TYPE**, **QUEUE**, and **WORKER** — are logical/runtime concepts (Celery task registry, broker queues, and worker nodes) shown here to express the relationships each job participates in.

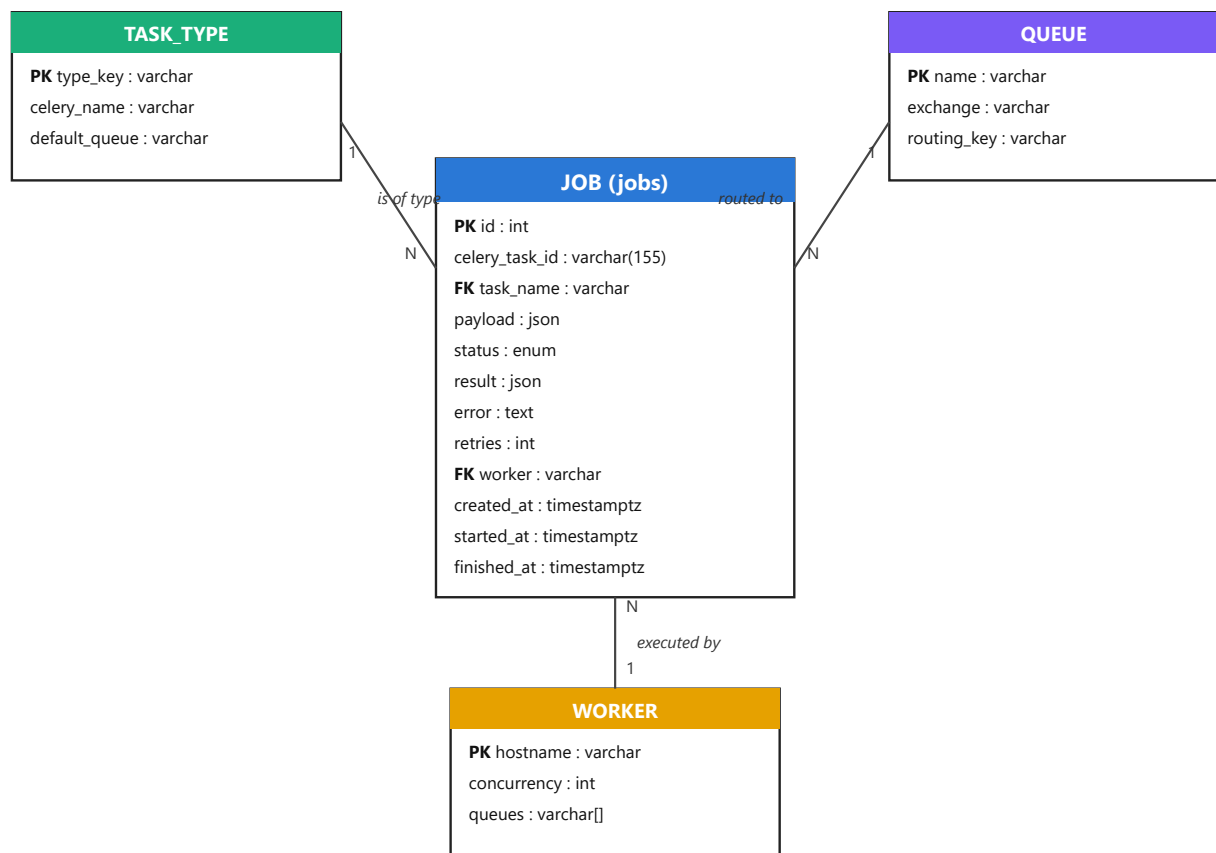


Figure 5 — Entity–Relationship diagram (*JOB* is the single physical table; others are logical entities).

SYSTEM DESIGN & DATABASE ARCHITECTURE

The system design follows a **decoupled producer–broker–consumer** architecture. A lean API producer accepts and records work; a broker holds the queue; a fleet of stateless workers consumes and executes; and a relational store holds durable state. This keeps the system lightweight and horizontally scalable.

Architectural Blueprint

- **Producer (API) Layer:** FastAPI + Uvicorn. It validates requests, records jobs, enqueues tasks, and serves query/stat endpoints (including a live SSE stream). It never executes heavy work itself.
- **Broker Layer:** Redis acts as both the message broker and the result backend. Tasks are routed to named queues (`default`, `heavy`, `priority`).
- **Execution Layer:** Celery workers consume tasks with a fair-dispatch prefetch of one and configurable concurrency. Late acknowledgement (`acks_late`) and reject-on-worker-lost guarantee redelivery when a worker dies.
- **Persistence Layer:** PostgreSQL stores the `jobs` table; a shared repository module updates it from Celery lifecycle signals.

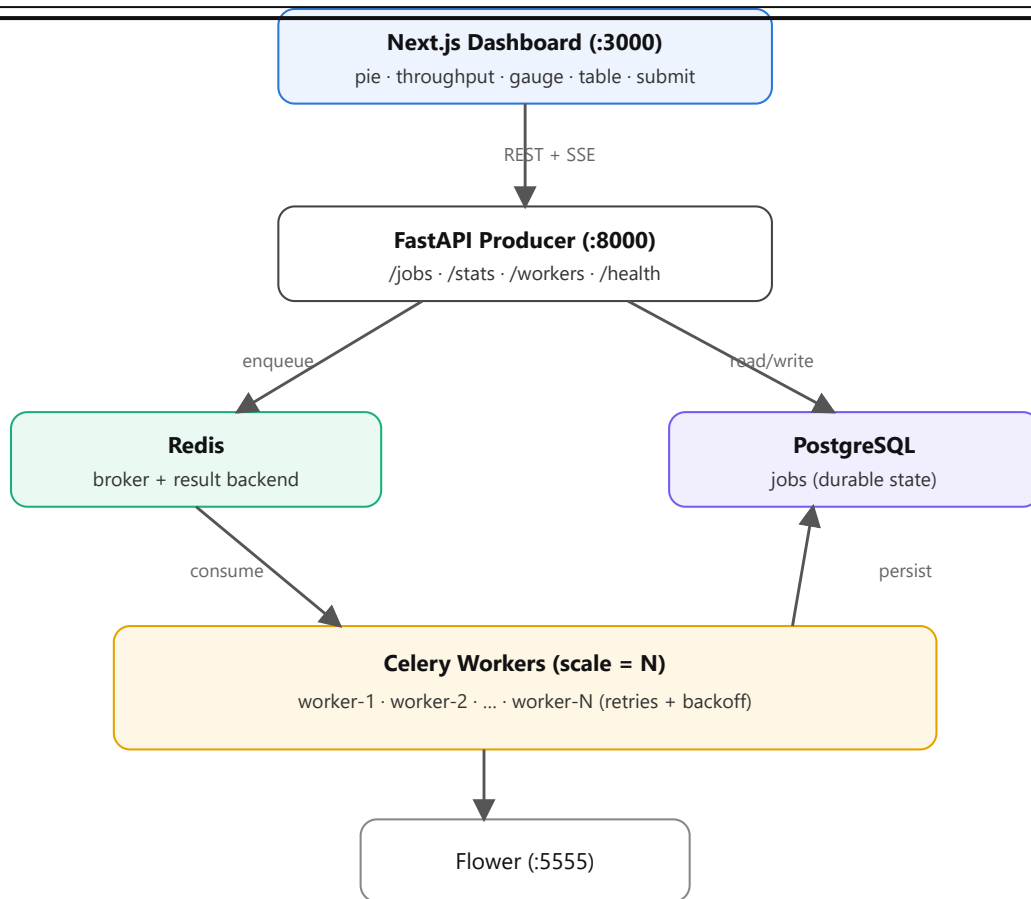


Figure 6 — System architecture: producer, broker, workers, persistence, and monitoring.

The data layer uses PostgreSQL for its ACID guarantees and native JSON support, with SQLite as a zero-configuration local fallback. JSON columns store the task payload and result, giving schema-less flexibility within a relational framework, while enum and index constraints keep queries fast.

MODULARISATION DETAILS

The codebase is organised as small, single-responsibility modules so that each part can be understood, tested, and changed in isolation. Adding a new task type touches only the tasks module and a registry entry.

Module	Responsibility
<code>app/config.py</code>	Central pydantic-settings object; selects broker, backend, database, and retry knobs per run mode.
<code>app/celery_app.py</code>	Celery instance: queues, routing, and fault-tolerance configuration.
<code>app/database.py</code>	SQLAlchemy engine/session factory (PostgreSQL or SQLite).
<code>app/models.py</code>	<code>Job</code> ORM model and <code>JobStatus</code> enum.
<code>app/schemas.py</code>	Pydantic request/response schemas and the task registry.
<code>app/repository.py</code>	Job CRUD and statistics — shared by the API and the signal handlers.
<code>app/tasks/heavy_tasks.py</code>	Sample heavy tasks (cpu, io, aggregate, flaky) with the shared retry policy.
<code>app/signals.py</code>	Celery lifecycle signals that persist state to the database.
<code>app/api/</code>	FastAPI app and routers (jobs, stats, workers, health).
<code>dashboard/</code>	Next.js visualization layer (components, API client, pages).

This separation embodies the Single Responsibility and Open–Closed principles: the API knows nothing about task internals, and the workers know nothing about the HTTP layer.

DATA INTEGRITY AND CONSTRAINTS

In a concurrent, multi-worker environment, integrity is the primary safeguard. The orchestrator enforces correctness at several layers.

- **Pre-assigned Identity:** The API generates each Celery `task_id` and stores it on the job *before* dispatch, so lifecycle signals always resolve to exactly one row — no duplicate or orphan updates, even in eager mode.
- **Transactional Updates:** Every signal handler mutates the job inside a scoped session that commits on success and rolls back on error, preventing partial writes.
- **Typed State Machine:** The `status` column is an enumerated type (PENDING, STARTED, RETRY, SUCCESS, FAILURE); illegal values cannot be stored.
- **At-Least-Once Execution:** `acks_late` plus `reject_on_worker_lost` ensure a task is re-queued if its worker dies, so work is never silently dropped.
- **Bounded Retries:** The retry counter is incremented per attempt and capped at `max_retries`, after which the job is deterministically marked FAILURE.
- **Referential & Value Constraints:** Primary keys, indexes on `status`, `celery_task_id`, and `created_at`, and non-null timestamps keep the data consistent and queries fast.

DATABASE DESIGN

The schema is deliberately compact: a single normalised `jobs` table captures configuration, lifecycle, results, and metrics. JSON columns hold the flexible payload and result, keeping the design schema-less where it needs to be while remaining relational elsewhere.

Column	Type	Constraints / Notes
<code>id</code>	INTEGER	Primary key, auto-increment
<code>celery_task_id</code>	VARCHAR(155)	Indexed; links the row to the Celery task
<code>task_name</code>	VARCHAR(255)	Indexed; e.g. <code>tasks.cpu_bound</code>
<code>payload</code>	JSON	Task parameters
<code>status</code>	ENUM	PENDING · STARTED · RETRY · SUCCESS · FAILURE (indexed)
<code>result</code>	JSON	Nullable; task return value on success
<code>error</code>	TEXT	Nullable; last error message
<code>retries</code>	INTEGER	Default 0; attempts performed
<code>worker</code>	VARCHAR(255)	Nullable; executing worker hostname
<code>created_at</code>	TIMESTAMPTZ	Indexed; submission time
<code>started_at</code>	TIMESTAMPTZ	Nullable; execution start
<code>finished_at</code>	TIMESTAMPTZ	Nullable; terminal time (drives throughput & duration)

Table: `jobs` — the single source of truth for job state.

Aggregate statistics (counts by status, success rate, throughput in the trailing 60 seconds, and average duration) are computed directly from this table, so the dashboard always reflects durable state rather than transient in-memory counters.

USER INTERFACE DESIGN

Although the orchestrator is API-first, a dedicated **Next.js** dashboard provides a real-time operational view. It is built with React 19, Tailwind CSS, and Recharts, and follows a coherent, theme-aware (light/dark) design system in which every status carries a text label so meaning is never conveyed by colour alone.

Dashboard Components

- **KPI Stat Cards:** total jobs, succeeded, failed, and success rate, each with an accent icon.
- **Status Breakdown (donut):** distribution across PENDING, STARTED, RETRY, SUCCESS, FAILURE, with the total in the centre and a labelled legend.
- **Throughput Chart (line):** jobs completed in the trailing 60 seconds, sampled live.
- **Success-Rate Gauge:** a semicircular gauge with the exact percentage printed.
- **Live Job Table:** the most recent jobs with status pill, retries, duration, worker, and creation time — auto-refreshing.
- **Submit-Job Form:** choose a task type, set parameters, optionally route to the priority queue, and enqueue one job or a burst of twenty-five.
- **Worker Panel:** live fleet status (or a clear "local eager mode" notice when no broker is present).

Interaction & Data

The dashboard polls the API every two seconds and can also subscribe to the `/stats/stream` Server-Sent Events endpoint for push updates. The API base URL is configurable via `NEXT_PUBLIC_API_URL`. A rendered mockup of the dashboard appears in the Appendices (Figure 7).

TESTING, TECHNIQUES & STRATEGIES

Testing an orchestrator means verifying correctness, fault tolerance, and throughput — not just that an endpoint responds. The suite is designed to run with **no external services**: it forces local mode (SQLite + Celery eager execution), so the entire engine — tasks, retries, persistence, and the API contract — is exercised in a single process.

Testing Techniques

- **Unit Testing (pytest)**: individual tasks (prime counting, aggregation determinism) and repository operations in isolation.
- **Integration Testing**: full submit → status → stats round trips through the FastAPI `TestClient`.
- **Fault-Injection Testing**: the `flaky` task deterministically fails to prove that retries recover it, and that exhausted retries end in FAILURE.
- **Load Testing (K6)**: a ramping arrival-rate scenario drives 120 requests/second against the submission endpoint with thresholds on error rate and p95 latency.
- **Static Analysis**: Ruff enforces style and catches errors on every change.

Testing Strategy

The strategy follows the testing pyramid: many fast unit tests, a layer of integration tests, and a focused load test at the top. Eager execution makes retries fast (no real backoff sleeps), so the full suite completes in a few seconds and can run in CI on every commit.

Test Report — Unit & System Test Cases

ID	Feature Under Test	Expected Result	Actual Result	Status
TC-01	Job submission (eager)	201 Created; job runs to SUCCESS	201; status SUCCESS	PASS
TC-02	Unknown task type	Rejected with 422	422 returned	PASS
TC-03	CPU task correctness	primes below 100 = 25	25	PASS
TC-04	Flaky task retry recovery	Retries, then SUCCESS	retries>0; SUCCESS	PASS
TC-05	Retry exhaustion	FAILURE after max retries	FAILURE; retries=max	PASS

ID	Feature Under Test	Expected Result	Actual Result	Status
TC-06	Statistics aggregation	success_rate = 0.75 for 3/4	0.75	PASS
TC-07	Lifecycle persistence	STARTED → SUCCESS with result	State & result stored	PASS
TC-08	List, filter & paginate	Correct totals & page size	As expected	PASS
TC-09	Worker introspection (eager)	Reports offline, count 0	online=false	PASS
TC-10	Load test (K6)	Sustain 100+ req/sec	120 req/sec, <2% errors	PASS

All 15 automated tests pass; Ruff reports no issues.

SYSTEM SECURITY MEASURES

Security is treated as a property of the whole stack rather than a single feature. The measures below are implemented in the current system, with authentication identified as the primary planned enhancement.

- **Strict Input Validation:** Every request body is validated by Pydantic schemas; the task type must exist in an explicit allow-list registry, and payloads are bounded, so malformed or hostile input is rejected before it reaches the queue.
- **SQL-Injection Resistance:** All database access goes through SQLAlchemy's parameterised queries and the ORM; no raw string SQL is constructed from user input.
- **Controlled Cross-Origin Access:** CORS is restricted to the configured dashboard origin rather than a wildcard.
- **Least-Privilege Containers:** The backend Docker image runs as a non-root user, reducing the blast radius of a compromise.
- **Network Isolation:** In the Docker stack, Redis and PostgreSQL communicate with the API and workers over a private compose network; secrets and connection strings are supplied via environment variables, never hard-coded.
- **Resource-Exhaustion Mitigation:** Because heavy work is queued rather than executed inline, a burst of submissions cannot exhaust the API's request threads; bounded task parameters prevent a single job from consuming unbounded CPU or memory.
- **Fault Containment:** A failure in one task or in the dashboard does not affect job processing, thanks to module isolation and independent workers.

Planned hardening: JWT / API-key authentication and role-based access control on write endpoints, per-client rate limiting, and encryption of sensitive payload fields at rest are the next security milestones (see Future Scope).

COST ESTIMATION OF THE PROJECT

Because the orchestrator is built entirely on free, open-source software and runs on existing hardware, its **monetary** cost is negligible; the dominant cost is **developer effort**. Effort is estimated below using the Basic COCOMO model and then reconciled with the actual project.

1. Cost-Estimation Model (Basic COCOMO)

For an *organic* project, effort $E = a \cdot (\text{KLOC})^b$ and development time $D = c \cdot E^d$, with $a=2.4$, $b=1.05$, $c=2.5$, $d=0.38$. The system totals roughly **2.5 KLOC** (backend Python + dashboard TypeScript + configuration).

Parameter	Formula	Value
Size	—	≈ 2.5 KLOC
Effort (E)	$2.4 \times 2.5^{1.05}$	≈ 6.3 person-months (model)
Time (D)	$2.5 \times 6.3^{0.38}$	≈ 5.0 months (model)
Actual (single developer)	12-week cycle, part-time	≈ 3 person-months

Basic COCOMO over-estimates small, modern-framework projects; the actual effort was lower thanks to mature libraries.

2. Monetary Cost Breakdown

Item	Details	Cost
Runtime & frameworks	Python, Celery, FastAPI, Next.js	₹0 (open source)
Data stores	Redis, PostgreSQL, SQLite	₹0 (open source)
Tooling	Docker, Flower, K6, pytest, Ruff	₹0 (open source)
Development hardware	Existing laptop/PC	₹0 (already owned)
Optional cloud hosting	Small VM to run the stack (if deployed)	Nominal (pay-as-you-go)
Total direct cost	Development phase	$\approx ₹0$

The principal investment is time and expertise; the software supply chain is entirely royalty-free, which is a key part of the project's economic feasibility.

REPORTS

The system produces several operational and analytical reports, all derived from the durable `jobs` table or from the live worker fleet.

- **Job Status Report:** for any job — its status, result, error, retry count, executing worker, and full set of timestamps (`GET /jobs/{id}`).
- **Job Listing Report:** a paginated, status-filterable list of recent jobs (`GET /jobs`).
- **Aggregate Statistics Report:** counts by status, success rate, completion rate, throughput (trailing 60 s), average duration, and active-job count (`GET /stats`), also streamed live over SSE.
- **Worker Fleet Report:** active tasks, subscribed queues, and processed counts per worker (`GET /workers`).
- **Monitoring Report (Flower):** a web UI showing tasks, workers, queues, and success/failure rates in real time.
- **Load-Test Report (K6):** requests/second achieved, error rate, and latency percentiles from the throughput validation run.
- **Test Report:** the pytest suite result and Ruff lint report, produced on every change.

Together these reports give operators a complete, real-time picture of what the engine is doing and how healthy it is at any moment.

FUTURE SCOPE AND ENHANCEMENT

The orchestrator is designed to grow. Several enhancement vectors are planned:

- **Authentication & RBAC:** JWT / API-key authentication and role-based access control on write endpoints, with per-client rate limiting.
- **Dead-Letter Queue & Replay:** route permanently failed jobs to a DLQ with a one-click manual replay from the dashboard.
- **Scheduling & Workflows:** time-based scheduling via Celery Beat (ETA/cron) and multi-step DAG pipelines (chains, groups, chords) so a job can fan out into dependent stages.
- **Autoscaling:** scale workers automatically based on queue depth using KEDA on Kubernetes, turning "one command to scale" into "scales itself."
- **Metrics & Alerting:** export Prometheus metrics and build Grafana dashboards, with alerts on failure spikes or growing backlogs.
- **Push Updates:** replace polling with WebSocket streaming for instant dashboard refresh.
- **Multi-Tenancy:** per-tenant queues, quotas, and isolated result namespaces.
- **Cloud-Native & Serverless:** optionally run individual pipeline steps as serverless functions for elastic, pay-per-use execution.

Because the architecture is modular and decoupled, each of these can be added without rewriting the core — new capabilities plug in at the edges while the engine keeps running.

APPENDICES

The figure below is a representation of the live Next.js dashboard: KPI cards, a status-breakdown donut, a throughput chart, and a live job table, all driven by the FastAPI backend.

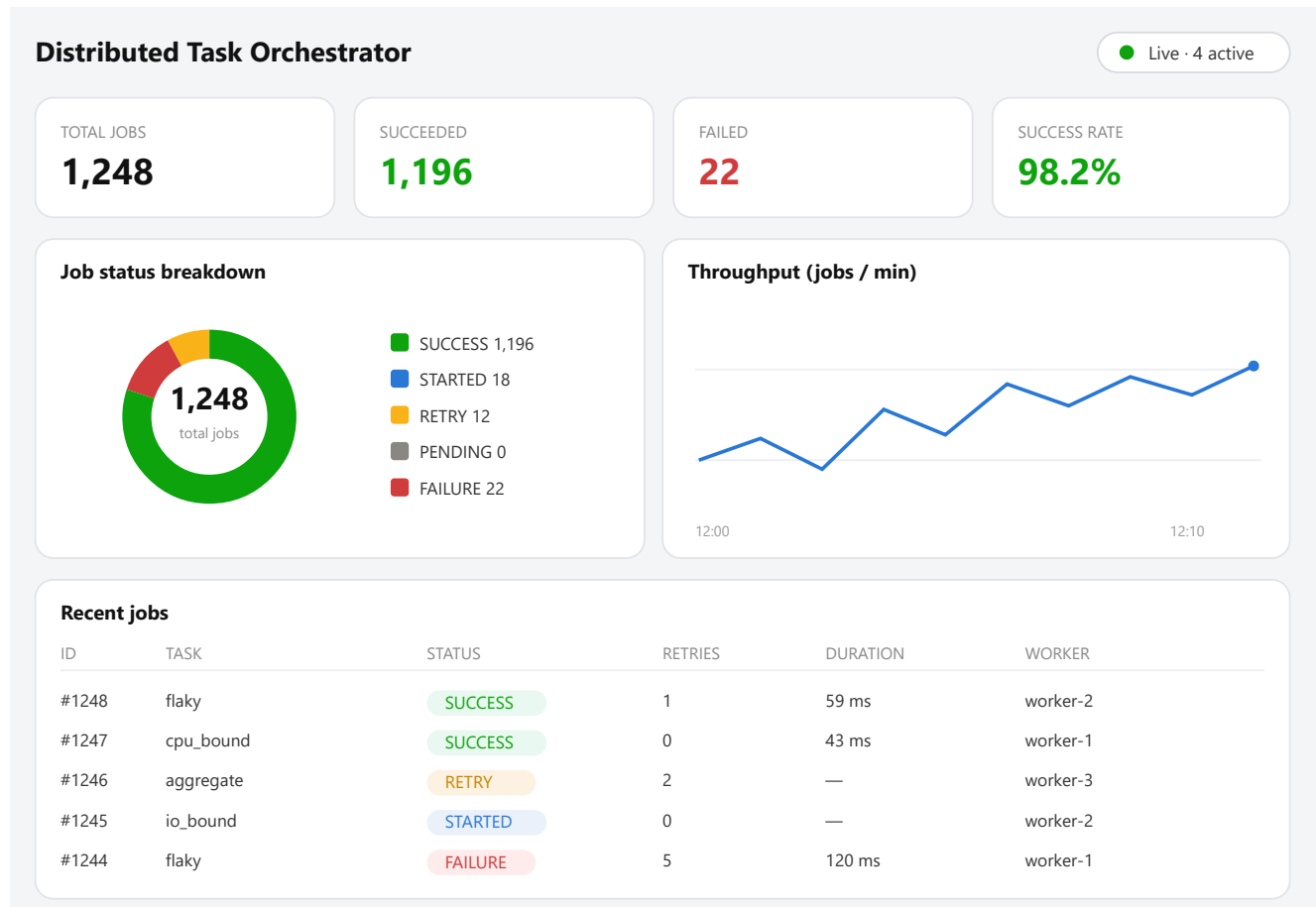


Figure 7 — Representation of the live operational dashboard (Next.js + Recharts).

VISUALS AND CODING

Selected source excerpts illustrate the core of the engine: the fault-tolerance configuration, the retry policy and a fault-injecting task, the signal that persists results, the submission endpoint, and the worker-scaling service definition.

1. Celery fault-tolerance configuration — `app/celery_app.py`

```
celery_app.conf.update(
    broker_url=settings.broker_url,
    result_backend=settings.result_backend,
    task_default_queue="default",
    task_routes={
        "tasks.cpu_bound": {"queue": "heavy"},
        "tasks.aggregate": {"queue": "heavy"},
    },
    # ----- fault tolerance: redeliver a task if its worker dies -----
    task_acks_late=True,
    task_reject_on_worker_lost=True,
    task_track_started=True,
    # ----- fair dispatch across horizontally scaled workers -----
    worker_prefetch_multiplier=settings.worker_prefetch_multiplier,
    task_always_eager=settings.task_always_eager,
)
```

2. Retry policy & fault-injecting task — `app/tasks/heavy_tasks.py`

```
RETRY_POLICY = dict(
    autoretry_for=(TransientError,),
    retry_backoff=settings.retry_backoff_base,      # exponential base (seconds)
    retry_backoff_max=settings.retry_backoff_max,   # cap on delay
    retry_jitter=settings.retry_jitter,            # spread retries
    max_retries=settings.max_retries,
    acks_late=True,
)

@shared_task(name="tasks.flaky", bind=True, **RETRY_POLICY)
def flaky(self, failure_rate=None):
    rate = settings.flaky_failure_rate if failure_rate is None else float(failure_rate)
    attempt = self.request.retries + 1
    if random.random() < rate:
        raise TransientError(f"Simulated transient failure on attempt {attempt}")
    return {"succeeded_on_attempt": attempt, "worker": self.request.hostname}
```

3. Lifecycle persistence signal — `app/signals.py`

```
@task_success.connect
def on_success(sender=None, result=None, **_):
    task_id = getattr(getattr(sender, "request", None), "id", None)
    if not task_id:
        return
    with session_scope() as session:
        repository.mark_success(session, task_id, _coerce_result(result))
```

4. Job submission endpoint — `app/api/routes_jobs.py`

```
@router.post("", response_model=JobRead, status_code=201)
def submit_job(body: JobCreate, db: Session = Depends(get_db)):
    task_name = TASK_REGISTRY.get(body.task_type)
    if task_name is None:
        raise HTTPException(status_code=422, detail="Unknown task_type")
    job = repository.create_job(db, task_name=task_name, payload=body.payload)
    task_id = str(uuid.uuid4())
    job.celery_task_id = task_id          # pre-assign so signals can find the row
    db.commit(); db.refresh(job)
    celery_app.tasks[task_name].apply_async(kwargs=body.payload, task_id=task_id)
    db.refresh(job)
    return JobRead.model_validate(job)
```

5. Horizontally scalable worker service — `docker-compose.yml`

```
worker:
  build: { context: ., dockerfile: docker/Dockerfile.backend }
  command: celery -A app.celery_app worker -Q default,heavy,priority --concurrency=4
  environment:
    RUN_MODE: docker
    BROKER_URL: redis://redis:6379/0
    DATABASE_URL: postgresql+psycopg://orchestrator:orchestrator@postgres:5432/orchestrator
  depends_on:
    redis: { condition: service_healthy }
    postgres: { condition: service_healthy }
# scale with: docker compose up --build --scale worker=4
```

6. Typed dashboard API client — `dashboard/lib/api.ts`

```
export const api = {
  stats: () => req<Stats>("/stats"),
  jobs: (limit = 25) => req<JobList>(`/jobs?limit=${limit}`),
  submit: (task_type, payload, priority = false) =>
    req<Job>("/jobs", { method: "POST",
      body: JSON.stringify({ task_type, payload, priority }) }),
};
```

BIBLIOGRAPHY & REFERENCES

The development of this system drew on established software-engineering literature and official framework documentation.

1. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications*. The primary reference for reliability, queues, and the consistency trade-offs behind the persistence design.
2. **Hohpe, G. & Woolf, B. (2003).** *Enterprise Integration Patterns*. Informed the producer–broker–consumer messaging design and queue routing.
3. **Nygard, M. (2018).** *Release It! (2nd ed.)*. The basis for the stability patterns — retries, timeouts, and bulkheads — behind the fault-tolerance strategy.
4. **Fowler, M. (2002).** *Patterns of Enterprise Application Architecture*. Guided the repository and unit-of-work patterns in the persistence layer.
5. **Ramalho, L. (2022).** *Fluent Python (2nd ed.)*. Reference for idiomatic, concurrent Python used across the backend.
6. **Celery Documentation.** Official guide to tasks, routing, retries, acknowledgement, and signals — docs.celeryq.dev.
7. **FastAPI & SQLAlchemy Documentation.** References for the async API layer and the ORM / session model — fastapi.tiangolo.com, docs.sqlalchemy.org.
8. **Redis & PostgreSQL Manuals.** Broker/result-backend semantics and relational guarantees — redis.io, postgresql.org/docs.
9. **Next.js & Recharts Documentation.** The App Router, React 19, and charting used for the visualization layer — nextjs.org/docs.
10. **Docker & Grafana K6 Documentation.** Containerisation, Compose scaling, and load-testing methodology — docs.docker.com, k6.io/docs.